

1. Project Concept

Project Name: IceTrack — Skating School Administration Platform

IceTrack is a web application that helps a local skating school move away from paper-based tracking. The platform serves three roles — Admins, Instructors, and Parents/Skaters — and digitizes class management, attendance, skill progression, and communication.

Technical Stack

Layer	Technology
Frontend	Next.js (deployed on Vercel)
Backend	Node.js / Express (deployed on Render)
Database	Supabase (PostgreSQL + Auth + Storage)
AI Coding Agents	Claude Code / Claude (claude.ai)

2. Vibe-Coding Tasks (Prompts to Build Phase 2)

The following 8 tasks are the specific Claude Code prompts I will use to build Phase 2 of IceTrack. Each is scoped small enough for a single AI agent session. Phase 2 goal: reach ~2/3 of the full app with core features functioning and security verified.

Task 1: Instructor Dashboard — Class List & Attendance

Prompt: Using Claude Code: On /instructor/dashboard, fetch and display all classes assigned to the logged-in instructor from the classes table. When the instructor selects a class, show the enrolled skaters. Add a session date picker and an attendance toggle (present/absent) per skater. Store each attendance record in an attendance table (id, class_id, skater_id, session_date, present). If a skater previously attended, pre-fill their status.

Test: Mark attendance for 3 skaters on 2 different session dates. Confirm rows appear correctly in Supabase attendance table.

Task 2: Instructor Dashboard — Per-Class Skill Check

Prompt: Using Claude Code: Below the attendance section on the instructor class view, display a skill checklist for that class's level (pulled from the skills table). For each skater, show one skill at a time with a checkbox. When checked, insert a row into skill_completions (id, skater_id, skill_id, completed_date, instructor_id, note). Add an optional text field for the instructor to leave a personal note. If a skill was already completed, show the date and lock the checkbox.

Test: Check off 2 skills for different skaters. Add a note to one. Verify skill_completions rows in Supabase. Confirm completed skills show as locked on reload.

Task 3: Parent Dashboard — Child Progress View

Prompt: Using Claude Code: On /parent/dashboard, fetch all children linked to the logged-in parent via the enrollments table. For each child, display a progress card showing: current level name, all skills for that level with checkmarks for completed ones (from skill_completions), completion dates, and any instructor notes. Below skills, show an attendance history list (session date + present/absent). Apply RLS so a parent can only view their own children's data.

Test: Log in as a parent with 2 enrolled children. Verify each child shows correct skills and attendance. Confirm a parent cannot query another parent's child data via Supabase directly.

Task 4: Admin Dashboard — Class Management CRUD

Prompt: Using Claude Code: On /admin/dashboard, build a Classes section. Show a table of all classes: Level, Instructor, Day, Time, Ice Location. Add a Create Class form: select level, select instructor (profiles where role=instructor), day of week, time slot, ice_location. On submit, insert into classes table. Add Edit and Delete buttons per row. Deleting a class should also remove related enrollments (cascade or manual cleanup).

Test: Create 2 classes, edit one, delete one. Confirm the deleted class's enrollments are also removed from Supabase.

Task 5: Admin Dashboard — Enrollment Management

Prompt: Using Claude Code: On /admin/dashboard, add an Enrollments section. Show all enrollments with columns: Skater Name, Level, Class Time, Parent Name. Add an Enroll Skater form: select parent, select skater (child of that parent), select class. Insert into enrollments. Add a Remove button per enrollment. Also add a make-up session flag (is_makeup boolean) to enrollments so admins can mark when a skater attends a different class due to an absence.

Test: Enroll 2 skaters into different classes. Mark one as a make-up. Remove one enrollment. Confirm all changes in Supabase.

Task 6: Unit Test — Skill Completion Logic

Prompt: Using Claude Code: Write a Jest unit test file at /tests/skillCompletion.test.ts. Test the following: (1) a skill can be marked complete for a skater and the correct row is returned, (2) marking the same skill twice does not create a duplicate (upsert behavior), (3) a skater from a different level cannot have a skill from another level marked complete. Mock the Supabase client for all tests. Run the tests and confirm they pass.

Test: Run npx jest in terminal. All 3 tests pass with green output. Screenshot the passing test results.

Task 7: Security Audit — Claude as Security Consultant

Prompt: Using Claude: Act as a security consultant for the IceTrack skating school app. Review the following areas and identify vulnerabilities: (1) Can a parent access another parent's child data via the Supabase client? (2) Can an instructor modify skill completions for a class they are not assigned to? (3) Are any API keys or secrets exposed in the frontend bundle? (4) Is there any route accessible without authentication? For each vulnerability found, suggest the exact RLS policy or middleware fix. Document what was changed and what was already safe.

Test: Apply any fixes Claude suggests. Re-test protected routes while logged out. Re-test cross-user data access. Document results in the Phase 2 security section.

Task 8: Ice Rink Location Map — Admin Marks, Users View

Prompt: Using Claude Code: Add an ice_location field to the classes table (text, e.g. "Zone A", "Zone B"). On /admin/dashboard, add a simple rink diagram (SVG grid divided into zones). Admin can click a zone to assign it to a class session. On /parent/dashboard and /instructor/dashboard, show the assigned zone for each upcoming class with a highlighted map so users can see where to go.

Test: Assign Zone B to a class as admin. Log in as a parent enrolled in that class and confirm the zone is visible and highlighted on their dashboard.

3. Economic Forecast (10 req/user/month)

Projected monthly infrastructure costs at three scale tiers assuming 10 requests per user per month and ~100 MB baseline image storage (profile photos, skill card stickers).

Metric	500 Users	5,000 Users	50,000 Users	Notes
Monthly Requests	5,000	50,000	500,000	10 req × users
Image Storage	100 MB	500 MB	5 GB	Profile + sticker images
Vercel (Frontend)	\$0	\$20	~\$35	Free tier: 100GB bandwidth
Render (Backend)	\$0	\$7	\$25	Free tier: 750 hrs/mo
Supabase (DB + Auth)	\$0	\$25	~\$45	Free: 500MB DB, 1GB storage
Total / Month	\$0 / mo	\$52 / mo	~\$105 / mo	

Free Tier Strategy:

- Vercel Free: Up to 100 GB bandwidth/mo, perfect for 500-user tier with no cost.
- Render Free: 750 compute hours/month covers a single always-on service at small scale.
- Supabase Free: 500 MB database storage, 1 GB file storage, 50k monthly active users — sufficient for Phase 1 and small pilot deployments.
- Scale trigger: Move to paid tiers when active users exceed ~1,000 or DB storage exceeds 400 MB.
- Storage note: Instructor skill card sticker images should be compressed to <50 KB each. At 50k users, profile images become the largest cost driver — consider compressing uploads to 100-200 KB max.

Cost References:

- Vercel Pricing: <https://vercel.com/pricing>
- Render Pricing: <https://render.com/pricing>
- Supabase Pricing: <https://supabase.com/pricing>

4. Testing Roadmap

The following tests verify that the Auth flow correctly connects the UI, Backend (Node/Supabase client), and Database (Supabase PostgreSQL). These serve as Phase 1 proof of connectivity.

Test	Action	Expected Result	Verified By
T1: Registration	Register a new Parent account via /register form	User row appears in Supabase Auth + profiles table with role=parent	Screen recording showing Supabase dashboard + row
T2: Role Redirect	Log in as Admin, Instructor, Parent separately	Each role lands on its specific dashboard URL	Screen recording of 3 login flows
T3: Protected Route	Visit /admin/dashboard while logged out	Redirect to /login page	Screen recording
T4: Logout	Click Logout on any dashboard	Session cleared; redirect to /login; back button does not restore session	Screen recording
T5: .gitignore	Inspect GitHub repo for .env files	No .env or .env.local files visible in repository	GitHub repo screenshot
T6: RLS Check	As Parent user, query Supabase for another user's profile directly	Row returned is empty; access denied by RLS policy	Supabase SQL editor screenshot

•